

UPWARDS Drone Project Handbook

Everything you need for the drone project, in one place. You are going to program a real drone to fly itself: find markers with its camera, fly to them, and land. You start on your laptop with a simulator, then earn a real flight.

How to use this handbook. Work top to bottom the first time (Set up, then How it works, then Part 1, 2, 3). After that, use the Quick index below to jump to what you need. Read it next to the code, and type the commands yourself.

Quick index

I want to...

I want to...	Go to
set up my laptop	Set up your laptop
understand how the project works and the flight rules	How this project works
look up a verb or a command	Cheat sheet
fly the drone by hand	Part 1.2
write the code that flies the drone	Part 1.4 and Part 2.1
make the drone fly itself to a marker	Part 1 and Part 2
build my own mission	Part 2.6
fly for real	Part 3.2
make my demo video and poster	Part 3.3
tune the drone's behavior	Appendix B: Settings
try the advanced labs	Going further

Something is wrong...

Symptom	Go to
it will not connect / hangs	Appendix A: Troubleshooting
it says "Run the simulator first"	How it works: flight rules
it says "Finish your code first"	How it works: flight rules
it flew but did nothing / the picture froze	Appendix A: Troubleshooting
<code>ModuleNotFoundError</code> on install	Set up your laptop

Contents: Set up · How it works · Cheat sheet · Part 1 Fly and See · Part 2 Lock On, Navigate, Decide · Part 3 Signal, Build, Share · Going further · Appendix A Troubleshooting · Appendix B Settings · Appendix C First-flight test (facilitators) · Appendix D All your files.

Set up your laptop

You need **Python 3.11 or 3.12** (3.10 through 3.13 all work). Get it from <https://www.python.org/downloads/>. On Windows, check **"Add python.exe to PATH"** on the first screen of the installer, and do **not** install Python from the Microsoft Store (that version

does not work here).

Windows, the fast way: double-click `setup_windows.bat` . It builds everything and checks it, then skip to the next section. If it closes instantly or cannot find Python, install Python as above and run it again.

Mac, the fast way: open the **Terminal** app. Type `cd` (with a space), drag this folder into the Terminal window so its path appears, press Enter, then run:

```
bash setup_mac_linux.sh
```

(Linux is the same command.)

By hand instead. Open a terminal in this folder and run these lines.

Windows:

```
py -m venv .venv
.venv\Scripts\activate
python -m pip install opencv-contrib-python matplotlib av pillow
python -m pip install --no-deps djitellopy
```

Mac or Linux:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install opencv-contrib-python matplotlib av pillow
python -m pip install --no-deps djitellopy
```

The last two lines are separate on purpose. Installing `djitellopy` with `--no-deps` keeps it from pulling a second copy of OpenCV that fights with the first. The `av` and `pillow` packages are what `djitellopy` uses to read the video, so they go in the first line.

Any time, check that a laptop is ready with `python check_setup.py` . It should say READY.

How this project works

Three parts, one goal: program a real drone to fly itself, then show your work. You build up six skills step by step (fly, see, lock on, navigate, decide, and signal), learn each from a worked example, and then invent your own mission.

The golden rule: test in the simulator first. Everything you write runs on your laptop with no drone (`python my_mission.py`). Only when it works in sim do you earn a flight slot.

The code you write lives in one file, `student_code.py` : five functions the drone uses to fly and decide. You fill them in. Until you finish one, the drone uses a safe backup, so it always flies safely, even with a wrong or missing function. Check your work any time, with no drone:

```
python check.py
```

Your job across the three parts is to make all of `check.py` pass with your own code, then watch the drone fly on the logic you wrote. In Part 2 you also write your own tests in `my_tests.py` .

The flight rules: how you earn a real flight

The drone will not fly for real until you have done the work. Two rules, both checked automatically:

1. **Run the simulator first, even for manual flight.** The drone refuses any real flight until you have run a mission in the simulator that flies to a marker (for example `python example_mission.py`). This proves your laptop, camera, connection, and mission work before a drone is in the air. If you skip it you will see "Run the simulator first."
2. **Finish your own code before an autonomous flight.** Before the drone flies *itself* to a marker, all five of your functions must be written (`python check.py` shows 5 of 5). If they are not, you will see "Finish your code first." Flying by hand (`fly_manual`) does not use those functions, so it skips this rule, but it still needs rule 1.

So in practice: you **fly by hand in Part 1** (right after the sim run in 1.1), and your drone **flies itself for real in Part 3**, once all five functions pass and you take your flight slot. A facilitator can turn either rule off in `settings.py` (`REQUIRE_SIM_FIRST` , `REQUIRE_OWN_CODE`), for example during the hardware test. You should leave them on.

Cheat sheet

Keep this open. Every mission has the same shape. Keep the `with` line: it guarantees the drone lands and your video is saved even if something goes wrong.

```
from mission_toolkit import Drone

with Drone(sim=True, name="my_mission") as drone:    # sim=True on laptop, False to fly
    drone.takeoff()
    drone.fly_to(10)
    drone.land_on(42)
```

Your verbs:

Verb	What it does
<code>drone.takeoff()</code> / <code>drone.land()</code>	Take off and hover / land where it is
<code>drone.search_for(id)</code>	Turn in place until it sees that marker. True/False
<code>drone.search_for_any([ids])</code>	Turn until it sees any of them. Returns the id found, or None
<code>drone.fly_to(id)</code>	Find the marker, then fly in and center on it. True/False
<code>drone.land_on(id)</code>	Fly to a marker, then land there
<code>drone.visit([ids])</code>	Fly to each marker in order
<code>drone.photograph("name")</code>	Save a photo of what it sees now
<code>drone.celebrate()</code>	A small safe wiggle
<code>drone.dance(["celebrate"])</code>	Run a short routine
<code>drone.say("text")</code>	Print a line in your drone's voice
<code>drone.decide(id)</code>	Ask your <code>decide_action()</code> what to do at a marker
<code>drone.do(action)</code>	Run the action word that <code>decide()</code> returned
<code>drone.see()</code>	The marker in view right now, or None
<code>drone.battery</code> / <code>drone.height</code>	Battery percent / height in cm
<code>drone.flip("f")</code>	A flip. Off unless a facilitator turns it on
<code>drone.fly_manual()</code>	Fly the real drone yourself with the keyboard (Part 1)

Run commands: - `python check.py` : test your five functions (no drone) - `python my_mission.py` : run your mission in the simulator - `python my_mission.py --real` : fly your mission for real - `python my_tests.py` : test your whole mission in the simulator - `python example_mission.py` : run the worked example in sim - `python webcam_marker_test.py` : see marker detection with a webcam - `python calibrate.py` : read the marker size at your hover spot (needs the drone, no takeoff) - `python plot_flight.py flightlogs/<folder>/telemetry.csv` : make your poster figure - Advanced: `python check_pid.py` , `python check_controls.py` , `python analyze.py --sim` , `python check_distance.py` , `python measure_distance.py`

The markers: 10 SAMPLE A, 20 SAMPLE B, 30 WAYPOINT 1, 40 WAYPOINT 2, 42 HOME BASE.

Part 1: Fly and See

Goal: fly a drone by hand, and understand how it finds markers.

1.1 Warm up in the simulator

Run `python example_mission.py` . Watch the whole mission play out as text. You just ran a full autonomous mission on your laptop, and you unlocked your first real flight (rule 1). Read `example_mission.py` and match each printed line to a line of code.

1.2 FLY: pilot it yourself

In your flight slot (your scheduled turn with a real drone), make a file called `manual.py` with these two lines, and run it from your terminal with `python manual.py` (`sim=False` already makes it a real flight, so you do not add `--real` here):

```
from mission_toolkit import Drone
with Drone(sim=False, name="manual") as drone:
    drone.fly_manual()
```

Keys: `t` takeoff, `w` / `s` forward/back, `a` / `d` left/right, `r` / `f` up/down, `z` / `x` turn, `l` land, `q` or `ESC` to quit. Click the video window first so the keys reach the drone. (Even this manual flight needs rule 1, the sim run: it confirms your laptop, camera, and connection work before a real drone is up.) This is the sense-decide-act loop with you as the brain.

1.3 SEE: how the drone finds things

Print the marker sheet (`markers/course_markers.pdf`). Run `python webcam_marker_test.py` and hold a marker to your camera. Watch the number called **side** grow as the marker gets closer. The drone judges distance from that number: bigger marker means closer.

Calibrate the hover distance. In your flight slot, set the drone on a box at the spot and height you want it to end up, pointing at a marker, and run `python calibrate.py` . Read the **side** number and put it in `settings.py` as `TARGET_SIDE_PX` . The drone never takes off for this.

1.4 WRITE THE CONTROL: make the drone fly itself in

Open `student_code.py` . Fill in three functions, running `python check.py` after each until it passes: - `clamp(value, low, high)` keeps a number inside a range. One line. - `centering_error(marker_x, frame_center)` is how far the marker is from the middle. One subtraction. - `steer_speed(error, gain, max_speed)` is the **proportional controller**: bigger error, bigger push. This is the function that actually flies the drone in toward a marker.

When `check.py` shows these three green, the drone is flying on YOUR code. Run `python example_mission.py` in sim. It looks the same as before, because your code now matches the backup it was using. `check.py` going green is your proof. You will see your code change things in the next step, when you tune the gain.

1.5 TUNE IT

Your `steer_speed` multiplies the error by a gain, `KP_SIZE` in `settings.py`. Try `KP_SIZE` at 0.02, then 0.10, then 0.40, running `python example_mission.py` each time and reading the first "Smooth / Slow / Wobbly" line (the one for marker 10). Too low is slow (0.02), a middle value is smooth (0.10), too high wobbles (0.40). Find a smooth value. (`KP_X` and `KP_Y` work the same way for left/right and up/down.) You will confirm it on a real flight in Part 3, once all five functions pass and you take your flight slot.

Checkpoint: `check.py` shows `clamp`, `centering_error`, and `steer_speed` passing, and you tuned the gain. **Stretch:** change `TARGET_SIDE_PX` so the drone stops closer or farther.

Part 2: Lock On, Navigate, Decide

Goal: make the control smart, and start building your own mission.

2.1 WRITE THE LOCK-ON CHECK

Back in `student_code.py`, fill in `is_aligned(err_x, err_y, err_size, tol_center, tol_size)`. It returns True only when the marker is centered (`err_x` and `err_y` are small) AND at the right distance (`err_size` is small). This is the function that decides when the drone has arrived and can stop. Run `python check.py` until it passes.

2.2 PID: smooth out the wobble (advanced, optional)

Your `steer_speed` is a **proportional** controller, the P in PID: the push is just the error times a gain. Set `KP_SIZE` to 0.40 and run the sim. It says "Wobbly," because a pure P controller overshoots. Real controllers add two more parts to settle it. This is the PID from Lecture 2: - **I** (integral) adds up the error over time, to close a small gap that never quite goes away. - **D** (derivative) reacts to how fast the error is changing, to stop the overshoot.

Build it: fill in the `PID` class in `pid_exercise.py`. You write one method, `update`. Test it with no drone:

```
python check_pid.py
```

When it passes, turn it on: set `USE_PID = True` in `pid_exercise.py`, keep `KP_SIZE = 0.40`, and add a small `KD_SIZE` (try 0.05). Run `python example_mission.py` in sim and watch the "Wobbly" become "Smooth". You just used D to calm the overshoot. You will fly on your PID for real in Part 3, once all five functions pass. **Further stretch:** add a little `KI_SIZE`, and read about "windup" (an I term that grows too large).

New to Python? An `f-string` like `f"sample_{found}"` builds text with a value dropped in. `import settings` at the top lets you read values like `settings.BASE_MARKER`. That is all you need.

2.3 NAVIGATE: visit markers in order

`drone.visit([30, 40])` flies to each marker in turn. Try your own route. This is a **state machine**: the drone is in one state (going to 30), finishes, then moves to the next.

2.4 WRITE THE DECISION

Fill in `decide_action(marker_id)` in `student_code.py`. It returns one word for what the drone should do at a marker:

"photograph", "celebrate", "dance", or "nothing". This one is yours to design, but end with `else: return "nothing"` so every marker gets a valid answer (`check.py` tests all five markers). In your mission, use it like this:

```
found = drone.search_for_any([10, 20])
if found:
    drone.fly_to(found)
    drone.do(drone.decide(found)) # runs your decide_action
```

To test both choices with no drone, edit `SIM_WORLD_MARKERS` in `settings.py` (remove 10 so it finds 20).

2.5 Read the worked example

Open `example_mission.py` and read every line. You now understand all of it, including the `steer_speed` and `decide_action` that you wrote.

2.6 Start your mission

Copy the idea into `my_mission.py`. Pick a theme (rescue, Mars scout, delivery, anything). Build it in the simulator and run it over and over. Keep the `mission(drone)` structure and the `with Drone(...)` line.

2.7 TEST BEFORE YOU FLY

Open `my_tests.py`. It already has a few tests written for you: some check your functions, and one runs your whole mission in the simulator and confirms it takes off, returns to base, and lands. Run them:

```
python my_tests.py
```

Then add at least one test of your own at the bottom (there are ideas in the comments). Testing on the laptop means you never waste a flight slot on a bug you could have caught here.

Checkpoint: `check.py` and `my_tests.py` are both green, and you have a working mission in sim. **Stretch:** use `drone.see()` and an `if` to make a decision mid-flight. Add a `drone.dance(["celebrate"])`.

Part 3: Signal, Build, Share

Goal: add personality, fly it for real, and make your demo and poster.

3.1 SIGNAL: give your drone character

Use `drone.say(...)` to narrate, `drone.photograph("evidence")` to collect proof, and `drone.celebrate()` to react. `drone.flip("f")` is off by default; ask a facilitator if you want it enabled.

3.2 Before you fly: the checklist

Finish and polish your mission in the simulator. Then do not run `--real` until every box below is true. This is how you avoid wasting a flight slot on a bug you could have caught on the laptop. (The drone also checks the first two for you: see [the flight rules](#).)

- [] `python check.py` is all green (your five functions work)
- [] `python my_tests.py` is all green (your whole mission runs in sim)
- [] You calibrated `TARGET_SIDE_PX` for the real room with `python calibrate.py`
- [] Battery above 70 percent, VPN off, ethernet unplugged, on the `TELLO` WiFi
- [] Safety glasses on, the cage is closed, and your hand is on q or ESC

Run it from your terminal (not an IDE Run button). It will ask you to type `FLY` and press Enter to take off, then click the video window so q and ESC work. With every box checked, take your flight slot and fly.

3.3 Record your demo

Your demo is a short clip, about 60 to 90 seconds, that shows your drone doing its mission. Two pieces:

Piece 1: the drone's-eye video (automatic). Every real flight saves a video by itself. Look in `flightlogs/` for a folder named like `my_mission_20260728_143012/`. Inside is `flight.mp4`: the drone's camera view (FPV) for the whole flight, takeoff to landing, as a standard mp4 that opens in any video player. While the drone is chasing a marker it also draws the marker outline and the centering cross. Two things to know: recording only happens on a **real** flight (a sim run has no camera), and on a few laptops it saves `flight.avi` instead (it plays the same; the terminal tells you which it saved).

Piece 2: a video of the drone flying. Have a teammate film the drone in the cage with a phone. Get the takeoff, the search, and the landing.

Put them together (no editing software needed): play your phone video, then `flight.mp4`, back to back; or screen-record your laptop while the mission runs so the video window and the messages show together.

Tips: your `drone.say(...)` lines print in the terminal, not on the video, so screen-record the terminal or read them aloud while you film. Keep it short, show your best run, and show the moment it locks on and lands, that is the payoff. Say what your drone is doing and why, in your theme's voice. Put a link to your demo on your poster (a QR code is a nice touch).

3.4 Make your figure

```
python plot_flight.py flightlogs/<your folder>/telemetry.csv saves flight_figure.png for your poster.
```

3.5 Build your poster

Your facilitator will give you the poster template. Fill in your mission, how it works, your figure and evidence photo, the limits you found, and what is next.

3.6 Showcase

Present your poster and play your demo at our reward ceremony on Aug 14th.

Going further (advanced, optional)

Finished everything? These two tracks go deeper. Each is real code you write, testable with no drone, and each produces a number or a chart for your poster.

A1 Measure your controller (P versus PID)

You have flown on P and on PID. Which is actually better? Engineers answer with numbers, not opinions. Open `controls_lab.py` and write two functions, `overshoot_percent` and `settling_time`. Test them:

```
python check_controls.py
```

Then see the numbers. First set `KP_SIZE = 0.40` and `KD_SIZE = 0.05` in `settings.py` (a gain that makes P overshoot), and run:

```
python analyze.py --sim
```

It compares P and PID with no drone and saves `analyze_figure.png`. You should see P overshoot much more than PID (roughly 22 percent vs 8 percent, and slower to settle). Change the gains and run it again to watch the numbers move. On real flights, point it at your logs to compare two real runs:

```
python analyze.py flightlogs/P_run/telemetry.csv flightlogs/PID_run/telemetry.csv
```

Put the two response curves and the numbers on your poster.

A2 Distance in metres (turn your camera into a ruler)

The drone judges distance by how big the marker looks. With a little geometry you can turn that pixel size into real metres. Open `distance_lab.py`, read the short explanation, and write `distance_meters`. Test it:

```
python check_distance.py
```

Then calibrate `FOCAL_PX` for your camera (the file walks you through it, one tape measure and one reading from `calibrate.py`), and check it live against a tape measure:

```
python measure_distance.py
```

(`measure_distance.py` uses a backup until you write your own, so it always shows a number.) Bonus: `target_side_px_for(1.5)` tells you the `TARGET_SIDE_PX` to set so the drone stops exactly 1.5 m away.

Appendix A: Troubleshooting

- **It says "Run the simulator first."** That is flight rule 1. Run a mission in the simulator once (`python example_mission.py`), then fly.
- **It says "Finish your code first."** That is flight rule 2. Make `python check.py` pass (5 of 5), then fly the autonomous mission. (Manual flight does not need this.)
- **Cannot connect / it hangs.** Turn off any VPN, unplug ethernet, and join the `TELLO-XXXXXX` WiFi. Click **Allow** on the Windows firewall popup (on a Mac, allow the local-network prompt). If the drone light stopped blinking it went to sleep; power it off and on.
- **It flew but did nothing.** The marker was probably not in view, or the light was too dim or glary. Check it in `python webcam_marker_test.py` first.
- **The picture froze and it landed.** That is the safety feature: if the video freezes, the drone lands itself. Reconnect and try again.
- **ModuleNotFoundError: av** means the install was incomplete. Re-run the two install lines in [Set up your laptop](#), including `av pillow`.

Appendix B: Settings you can tune

Open `settings.py`. Everything you are allowed to change lives there. Do not edit files inside `mission_toolkit/`.

- `KP_X`, `KP_Y`, `KP_SIZE` : the gains your `steer_speed` uses (left/right, up/down, forward/back).
- `TARGET_SIDE_PX` : how big a marker looks when the drone has "arrived" (your hover distance).
- `SEARCH_YAW_SPEED` : how fast it turns while looking for a marker.
- `CENTER_TOLERANCE_PX`, `SIZE_TOLERANCE_PX` : how close counts as "locked on".
- `SIM_WORLD_MARKERS` : which markers exist in the simulator (delete one to test a missing marker).
- `KI_*`, `KD_*` : stay 0 unless you build the PID (Part 2.2).
- `REQUIRE_SIM_FIRST`, `REQUIRE_OWN_CODE` : the flight rules. Leave both `True`. A facilitator may set them `False` for the hardware test below.

Appendix C: First-flight test (facilitators)

Run this once, on one drone, before students fly. It confirms the drone connects, flies on the code, and lands safely, and it checks the three things the simulator cannot: the feel of manual flight, the photo colors, and the video-freeze failsafe. Budget about 15 minutes.

Turn the gates off for this test. You are testing hardware, not student code, so set `REQUIRE_SIM_FIRST = False` and `REQUIRE_OWN_CODE = False` in `settings.py` first (set them back to `True` before handing laptops to students).

Safety first. Do this inside the closed cage, safety glasses on, one drone only, floor and space above clear. Keep a hand on `q` or `ESC`. A facilitator can press `e` to cut the motors (the drone drops, which is why it is only for inside the cage).

Before you start. - [] In `settings.py`, set `REQUIRE_SIM_FIRST = False` and `REQUIRE_OWN_CODE = False` (set both back to `True` before students use the laptops). - [] `python check_setup.py` says READY. - [] Battery above 70 percent. - [] Drone on, light blinking. - [] Laptop on the `TELLO-XXXXXX` WiFi. **VPN off. Ethernet unplugged.** - [] Print one marker from `markers/course_markers.pdf` and stand it up at about camera height.

Test A: connect and fly by hand. Make `manual.py` (the two lines in Part 1.2) and run `python manual.py`. Click the video window, then `t` takeoff, tap `w / s / a / d`, `l` land, `q` quit. - [] It connects and a video window opens. [] `t` takes off. - []

Movement feels smooth, not stuttery (knob: `LATCH_S` in `mission_toolkit/_real.py`). - [] `l` lands, `q` quits cleanly.

Test B: marker detection and colors. Set the drone on a box pointing at the marker, run `python calibrate.py` (no takeoff). - [] A green box locks on and a `side` number shows. Note it for `TARGET_SIDE_PX : ____` - [] Run a short mission so a photo saves, then open the newest `photo_*.jpg`. **Colors look correct** (sky is blue, not orange). If red and blue are swapped, tell us.

Test C: autonomous lock-on. Run `python example_mission.py --real`, type `FLY`, click the video window. - [] It takes off and turns to search. [] It flies in and locks on. - [] It photographs, then lands on the base marker. - [] **The recording works:** the terminal prints `Video saved: ...`, the file plays, and it covers the whole flight. - [] **Abort works:** on a second run, press `q` mid-flight. The drone lands right away.

Test D: emergency stop (optional). On a run, press `e`. The motors cut and the drone drops (safe in the cage). Only if you are comfortable, and only inside the cage.

Result. Date / drone tested: _ · **Tests A, B, C passed:** _ · Anything odd: ____

If connection fails, see [Troubleshooting](#) above, and the mentor package (`DRONE_TEST_RUNBOOK.md` and the facilitator guide).

Appendix D: All your files

- **You edit these:** `student_code.py` (your five functions), `my_mission.py` (your mission), `my_tests.py` (your tests), `settings.py` (the knobs). Advanced: `pid_exercise.py`, `controls_lab.py`, `distance_lab.py`.
- **You run these:** `check.py`, `example_mission.py`, `check_setup.py`, `webcam_marker_test.py`, `calibrate.py`, `plot_flight.py`, and the advanced `check_pid.py` / `check_controls.py` / `analyze.py` / `check_distance.py` / `measure_distance.py`.
- **Do not edit:** everything in `mission_toolkit/` (the drone's brains and safe backups) and `markers/`.
- **Made for you when you fly:** `flightlogs/<name>_<time>/` with `flight.mp4`, `telemetry.csv`, and photos.